VOLUME 5

Turn Remote Data into an Experience

GitHub requests, portfolio mapping, Finder navigation, code preview, caching, fallbacks, and security

A beginner-to-engineer rebuild guide grounded in the real repository

PATTERN > PRACTICE > REPETITION > REFINEMENT > HABIT

Learning Map

Use this volume as a build session, not passive reading. Type the examples, pause at checkpoints, and keep the architecture questions beside your editor.

Chapter	You will learn
1. External API boundary	Fetch public GitHub data without leaking secrets.
2. Portfolio data boundary	Map remote responses into stable product entities.
3. Finder navigation	Separate UI effects from pure open-item policy.
4. Code preview	Coordinate explorer, editor, and remote file content.
5. Reliability and growth	Handle rate limits, caching, tests, and future backend needs.

Chapter 1: The External API Boundary

Chapter outcome: You will isolate browser requests and know when public client-side fetching is appropriate.

Actual GitHub request module

```
 */

const BASE_URL = "https://api.github.com";

const requestGitHub = async (url) => {
  return fetch(url);
};

// ============================================
// FETCH USER REPOS
// ============================================
export const fetchRepos = async () => {
  try {
    const username = import.meta.env.VITE_GITHUB_USERNAME;
    if (!username) return [];

    const res = await requestGitHub(`${BASE_URL}/users/${username}/repos`);

    if (!res.ok) return [];

    const data = await res.json();
    return Array.isArray(data) ? data : [];
  } catch (err) {
    console.error("fetchRepos error:", err);
    return [];
  }
};

// ============================================
// FETCH REPO TREE RECURSIVELY
// ============================================
export const fetchRepoTree = async (
  owner,
  repo,
  path = "",
  depth = 0
) => {
  if (depth > 3) return [];

  try {
    const res = await requestGitHub(
      `${BASE_URL}/repos/${owner}/${repo}/contents/${path}`
    );

    if (!res.ok) return [];

    const data = await res.json();
    if (!Array.isArray(data)) return [];

    const result = await Promise.all(
      data.map(async (item) => {
        if (!item?.name || !item?.type) return null;
```

```
        // ============================
        // FOLDER
        // ============================
        if (item.type === "dir") {
          return {
            id: item.sha,
            name: item.name,
            kind: "folder",
            type: "folder",
            icon: "/images/folder.png",
            path: item.path,
            repoName: repo,
            repoOwner: owner,
            children: await fetchRepoTree(owner, repo, item.path, depth + 1),
          };
        }

        // ============================
        // FILE
        // ============================
        const ext = item.name.includes(".")
          ? item.name.split(".").pop().toLowerCase()
          : "";

        return {
          id: item.sha,
          name: item.name,
          kind: "file",
          type: "file",
          icon: "/images/file.png",
          fileType: ext,
          path: item.path,
          download_url: item.download_url || "",
          repoName: repo,
          repoOwner: owner,
        };
      })
    );

    return result.filter(Boolean);
  } catch (err) {
    console.error("fetchRepoTree error:", err);
    return [];
  }
};
```

- VITE_GITHUB_USERNAME is public configuration and can safely appear in the browser bundle.
- A GitHub token must not be placed in a VITE_ variable because users can inspect it.
- Non-OK responses return empty arrays so the portfolio can fall back gracefully.
- Recursive tree loading stops after depth 3 to control request count and payload size.
- The API layer returns plain objects; it never opens windows or changes React state.

When a backend is required: Use a server route for private repositories, secret tokens, stronger caching, higher rate limits, auditing, or data normalization shared by many clients.

Chapter 2: The Portfolio Data Boundary

Chapter outcome: You will transform unstable remote data into a stable Finder-friendly model.

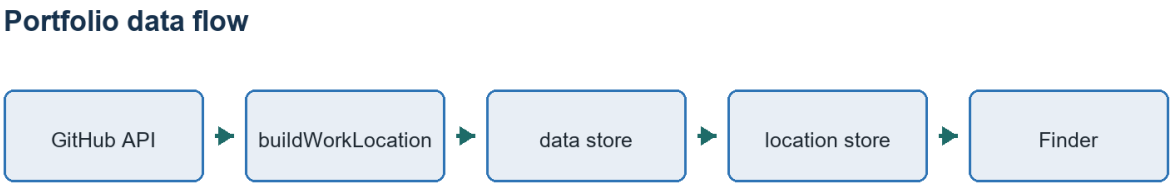

Figure 1. The mapper protects the interface from remote response shape changes.

Mapping repositories to Finder nodes

```
      description: "Frontend portfolio projects and UI experiments.",
      html_url: `https://github.com/${username}?tab=repositories`,
      homepage: "",
      owner: { login: username },
    },
    {
      id: "fallback-react-projects",
      name: "React Projects",
      description: "React practice projects available from GitHub.",
      html_url: `https://github.com/${username}?tab=repositories`,
      homepage: "",
      owner: { login: username },
    },
  ];
};

const attachParents = (nodes = [], parent = null) => {
  return nodes.map((node) => {
    const newNode = {
      ...node,
      parent,
    };

    if (node.children) {
      newNode.children = attachParents(node.children, newNode);
    }

    return newNode;
  });
};

const buildRepoFolder = async (repo) => {
  if (!repo.name) return null;

  let tree = [];

  try {
    tree = await fetchRepoTree(repo.owner.login, repo.name);
  } catch {
    tree = [];
  }
```

```
  const sourceFolder = {
    id: `${repo.id}-source`,
    name: "Source Code",
    icon: "/images/folder.png",
    kind: "folder",
    children: tree,
  };

  return {
    id: repo.id,
    name: repo.name,
    icon: "/images/folder.png",
    kind: "folder",
    children: [
      sourceFolder,
      {
        id: `${repo.id}-live`,
        name: "Live Site",
        icon: "/images/safari.png",
        kind: "file",
        fileType: "url",
        href: repo.homepage || repo.html_url,
      },
      {
        id: `${repo.id}-txt`,
        name: "Project.txt",
        icon: "/images/txt.png",
        kind: "file",
        fileType: "txt",
        description: [repo.description || "No description available."],
      },
    ],
  };
};

export const buildWorkLocation = async () => {
  if (cachedWork) return cachedWork;

  const repos = await fetchRepos();
  const sourceRepos = repos.length > 0 ? repos : createFallbackRepos();
  const repoFolders = await Promise.all(sourceRepos.map(buildRepoFolder));

  const rawWork = {
    id: 1,
    type: "work",
    name: "Work",
    icon: "/icons/work.svg",
    kind: "folder",
    children: repoFolders.filter(Boolean),
  };

  const work = attachParents([rawWork])[0];

  if (work.children.length > 0) {
    cachedWork = work;
  }

  return work;
};
```

buildRepoFolder creates a product concept from a repository: Source Code, Live Site, and Project.txt. attachParents adds navigation links after the tree is built. cachedWork prevents repeated network work during the same module lifetime. The output contract stays useful even if GitHub is replaced later.

Hook coordinating status and storage

```
import { useCallback } from "react";

import { useDataStore } from "#store";
import { buildWorkLocation } from "../utils/buildWorkLocation";

const hasProjectChildren = (work) => Boolean(work?.children?.length);

export const usePortfolioFileSystem = () => {
  const {
    work,
    status,
    error,
    setLoading,
    setWork,
    setError,
  } = useDataStore();

  const loadPortfolioFileSystem = useCallback(async () => {
    const currentWork = useDataStore.getState().work;
    if (hasProjectChildren(currentWork)) return currentWork;

    setLoading();

    try {
      const nextWork = await buildWorkLocation();
      setWork(nextWork);
      return nextWork;
    } catch (err) {
      setError(err);
      throw err;
    }
  }, [setError, setLoading, setWork]);

  return {
    work,
    status,
    error,
    isLoading: status === "loading",
    isLoaded: status === "ready",
    loadPortfolioFileSystem,
  };
};
```

Three Ways To Understand It

Beginner explanation: The API gives raw boxes; the mapper labels and arranges them for Finder.

Developer explanation: The hook coordinates asynchronous loading while the utility owns transformation.

Engineering explanation: A stable anti-corruption boundary prevents vendor response shapes from spreading through UI code.

Chapter 3: Finder Navigation

Chapter outcome: You will build navigation state, breadcrumbs, grids, and file-open behavior with clear ownership.

Finder coordinator

```
import FinderSidebar from "./components/FinderSidebar";
import {
  FINDER_OPEN_ACTIONS,
  getFinderOpenAction,
} from "./utils/getFinderOpenAction";

const Finder = () => {
  const { openWindow } = useWindowStore();

  const {
    activeLocation,
    setActiveLocation,
    goBackLocation,
  } = useLocationStore();

  if (!activeLocation) {
    return (
      <div className="flex items-center justify-center w-full h-full">
        Loading...
      </div>
    );
  }

  const openItem = (item) => {
    const action = getFinderOpenAction(item);

    if (action.type === FINDER_OPEN_ACTIONS.SET_LOCATION) {
      setActiveLocation(action.location);
      return;
    }

    if (action.type === FINDER_OPEN_ACTIONS.OPEN_WINDOW) {
      openWindow(action.windowId, action.data);
      return;
    }

    if (action.type === FINDER_OPEN_ACTIONS.OPEN_EXTERNAL_LINK) {
      window.open(action.href, "_blank");
    }
  };

  const favorites = [
    activeLocation?.type === "work" ? activeLocation : locations.work,
    locations.about,
    locations.gallery,
    locations.resume,
  ];
```

```
  const projects =
    activeLocation?.type === "work" ? activeLocation.children : [];

  return (
    <>
      <div id="window-header">
        <WindowControls target="finder" />
        <button type="button" onClick={goBackLocation} className="ml-3">
          <ArrowLeft className="icon" />
        </button>
        <Search className="icon" />
      </div>

      <FinderBreadcrumbs
        activeLocation={activeLocation}
        onSelectLocation={setActiveLocation}
      />

      <div className="bg-white flex h-full">
        <FinderSidebar
          favorites={favorites}
          projects={projects}
          activeLocation={activeLocation}
```

FinderWindow coordinates four things: current location, history action, presentation components, and execution of an open-item command. FinderSidebar, FinderBreadcrumbs, and FinderGrid stay focused on rendering and user events.

Pure file policy

```
import { isCodeFile, isExternalLinkFile } from "./fileTypes";

export const FINDER_OPEN_ACTIONS = {
  NONE: "none",
  SET_LOCATION: "set-location",
  OPEN_WINDOW: "open-window",
  OPEN_EXTERNAL_LINK: "open-external-link",
};

export const getFinderOpenAction = (item) => {
  if (!item) {
    return { type: FINDER_OPEN_ACTIONS.NONE };
  }

  if (item.fileType === "pdf") {
    return {
      type: FINDER_OPEN_ACTIONS.OPEN_WINDOW,
      windowId: "resume",
    };
  }

  if (item.kind === "folder") {
    return {
      type: FINDER_OPEN_ACTIONS.SET_LOCATION,
      location: item,
    };
  }

  if (item.fileType === "txt" && !item.download_url) {
    return {
```

```
    type: FINDER_OPEN_ACTIONS.OPEN_WINDOW,
    windowId: "txtfile",
    data: item,
  };
}

if (item.fileType === "img") {
  return {
    type: FINDER_OPEN_ACTIONS.OPEN_WINDOW,
    windowId: "imgfile",
    data: item,
  };
}

if (isExternalLinkFile(item.fileType) && item.href) {
  return {
    type: FINDER_OPEN_ACTIONS.OPEN_EXTERNAL_LINK,
    href: item.href,
  };
}

if (isCodeFile(item.fileType)) {
```

Node	Action	Result
folder	SET_LOCATION	Finder navigates into children.
pdf	OPEN_WINDOW resume	Resume viewer opens.
local txt	OPEN_WINDOW txtfile	Text viewer receives item data.
image	OPEN_WINDOW imgfile	Image viewer receives item data.
url/link	OPEN_EXTERNAL_LINK	Browser opens a new tab.
code extension	OPEN_WINDOW vsCode	Code preview receives selected file.
unknown	NONE	No side effect.

Chapter 4: Code Preview

Chapter outcome: You will understand how Explorer, Editor, Terminal, and the window coordinator divide work.

File	Responsibility	State
CodePreviewWindow.jsx	Coordinate selected file, loading, content, and shell layout.	Selected file and fetched content.

File	Responsibility	State
Explorer.jsx	Render a recursive file tree and report selection.	Mostly presentation.
Editor.jsx	Display source through Monaco with language selection.	Editor presentation/settings.
Terminal.jsx	Provide a decorative terminal/status region.	Minimal local UI.
getFileIcon.jsx	Map extensions to icon components.	Pure decision.

Performance: Monaco and React PDF are substantial packages, which is why their containing windows are lazy-loaded. Performance work begins with loading boundaries before micro-optimizing small functions.

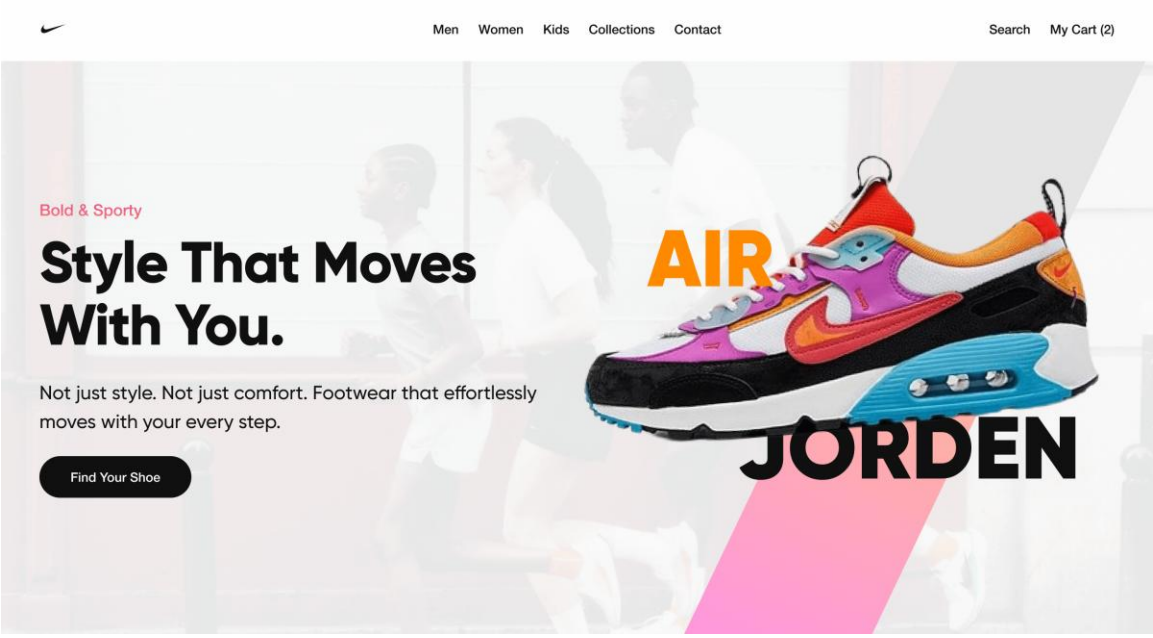

Figure 2. Project media is product content; keep it in public assets and reference stable paths.

Chapter 5: Reliability and Growth

Chapter outcome: You will know which parts are portfolio-sized today and how they would evolve under real traffic.

Concern	Current choice	Growth path
Rate limits	Unauthenticated public GitHub calls.	Server cache and authenticated server requests.

Concern	Current choice	Growth path
Caching	Module-level cached Work tree.	Query cache with expiry and background refresh.
Errors	Empty arrays and fallback repositories.	Typed errors, retry controls, monitoring.
Testing	Pure decisions are testable by design.	Unit tests for mappers/actions; integration tests for windows.
Large repositories	Depth limit of 3.	Lazy-expand folders and fetch on demand.
Security	No secret browser token.	Server-side credentials and policy checks.

Checkpoint

- Beginner: Explain why a VITE token is not secret.
- Intermediate: Design a test matrix for getFinderOpenAction.
- Advanced: Plan an on-demand folder-loading API for repositories with 100,000 files.